02-614 Week 10
String Algorithms

Aidan Jan

November 10, 2026

Applications of Wavelet Trees

Application 1: Document Retrieval

Given documents $D_1, \ldots, D_m$, and given a word w, return which documents contain w.

- A document is a list of words. i.e., strings over $\Sigma = \{\text{words}\}$. that is, an entire English word is a symbol in our alphabet.

For this problem, we can first create a string S, which is all the documents concatenated with unique terminator characters between them. i.e.,

$$S := D_1\$_1 D_2\$_2 \ldots D_m\$_m$$

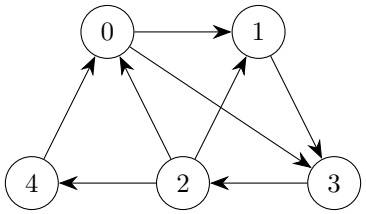
Pseudocode:

```
i = 1
while (i <= |S|):
    p_i = Select_w(i)       // Find the i-th occurance of our word
    d_i = Rank_$(p_i) + 1
    output d_i
    p' = Select_$(d_i)      // Get end of document with i-th word
    i = Rank_w(p') + 1      // Skip past the word occurrence in current document
```

- Recall that each word is a character in S. Therefore, $|S|$ refers to the total number of words across all documents.
- Wavelet trees are used to implement Select and Rank. The $|\Sigma|$ would be the size of the English dictionary.
- Runs in $O(\log |\Sigma|)$. The number of loops is equal to the number of documents output.

Application 2: Storing graphs

Wavelet trees can also be used to store graphs. Consider the following graph:



```
Node | Out-neighbors
0    | 1, 3
1    | 3
2    | 0, 1, 4
3    | 2
4    | 0
```

To store a graph, we basically need two functions:

- succ(u, i) = i-th node pointed to by u
- pred(u, i) = i-th predecessor (parent) of u

We can first make a data structure:

- $A = [1, 3, 4, 0, 1, 2, 0]$
- $B = [1, 0, 1, 1, 0, 1, 1]$

A represents outneighbors of nodes in clockwise order (This ordering does not really matter.), with node 0 first, then node 1, etc., all concatenated into an array. B stores where a node's adjacency list begins.

- A is stored in a wavelet tree while B is an RRR vector.
- The number of 1's in B always represents the number of nodes in the graph. The index of a 1 in B represents the index that the i-th adjacency list in A starts.

Now, we can implement the functions:

- succ(u, i) :=

```
p = Select_i (B, u + 1)     // O(1)
return A[p + i]             // O(log |E|)
```

- pred(u, j) :=

```
p = Select_u(A, j)          // O(log |E|)
return rank_1(B, p)         // O(1)
```

Note that $|E|$ is also the length of the lists.

What happens if we have a node with zero out-degree?

- There are two options: we can either include a special character (e.g., -1) to denote that a node has no out-neighbors
- Or, instead of using an RRR-vector as the B-array, we can just have one array with \$ between each adjacency list. The implementation of the algorithm would be similar to Application 1 in this case.

FM-Index

First, recall the BWT search algorithm. Given S and T, we want to find where S appears in T.

- To do this, we first do BWT on T. Then, we calculate the left column. We now have LF-mapping.
- Now, search S backwards, looking for the the character on the right side. Use the LF-mapping to search the left side. Repeat until entire string is found.

- The only thing is that we somehow need to know where the i-time the character appears on the right.
  - **Option 1:** This happens to be the $\text{rank}_c()$ function, which wavelet trees are amazing for.
  - **Option 2:** Use the FM-Index.

$$\tilde{T} = \text{Huffman}(RLE(MTF(BWT(T))))$$

    * BWT = Burrows Wheeler Transform
    * MTF = Move to front compression
    * RLE = Run length encoding
    * Huffman = Huffman encoding

For Option 2, we just have to make this super-compressed string searchable. First, consider the BWT of T.

- We break the $BWT(T)$ into blocks of u characters. With this representation, if we also break the compressed string $\tilde{T}$ into blocks such that each block in $\tilde{T}$ corresponds to a block in $BWT(T)$, the blocks might not be the same size, but they will be in the same order.
- Now, if $\tilde{T}$ is represented in terms of blocks $BZ_1, BZ_2, \ldots, BZ_b$, where b is the number of blocks in total, we can now represent this as *super-blocks*. A super-block is a block containing u blocks.
- Now we can start storing stuff. Make an array of size $|\Sigma|$ (one array index per alphabet character) *per superblock*, and it stores the number of occurrences of the given character up to and including the superblock b.
- Make another set of arrays of size $|\Sigma|$, one for each block in $BWT(T)$, and have each index store the number of characters up to the end of the block in $BWT(T)$, since the end of the last superblock.
- To search the rank of a character, first check its occurrence in its block, then check the number of occurrences in the previous superblocks.

How big is this data structure?

- We have a total of n/u block (excluding superblock) arrays. This takes advantage that there are u characters per block in BWT of T.
- Recall that each block array stores counts of letters. Therefore, every count must be smaller than u^2.
- Therefore, we need a total of $\frac{n}{y}\left(\log u^2\right)|\Sigma|$ bits to store the block array. If we assume $u = O(\log n)$, then the space taken up by block arrays is $O(\frac{n}{\log n}\log\log n)$. This approaches zero when n is large!
- Now for superblocks, we have a total of $(\frac{n}{u})/u$ total superblocks. Size of each super block array is $|\Sigma|\log n$. Calculating the bits required for the superblock, we get $n/\log n$. This is even smaller than the block arrays.

Computing Rank

Note that $\text{Rank}_c(i)$ is the sum of the following:

- Count c up through previous super blocks
- Count of c since end of that super block
- Count of c from block start to i

For the last point (count of c from block start to i), we can use the same trick as Four Russian's speedup. We build a table with all the counts precomputed. Since block sizes are $u = O(\log|T|)$, they really are not that big. Let

$$M[BZ_j, MTF_j, c, \hat{i}]$$

represent the count of c from block start to i.

- BZ_j is the super block index
- MTF_j is the move-to-front list (i.e., the current state of the move to front vector) at the start of BZ_j. This is necessary since MTF distorts the ordering of the numbers, but we can reconstruct the original ordering if given the state.
- c is character of interest
- $\hat{i}$ is the index in the local block

What is the size of the entire table?

$$\text{Size} = O(|\Sigma|^u \times |\Sigma|! \times |\Sigma| \times u \times \log u)$$

bits. The first four terms are the sizes of each dimension, and the actual value can be stored in $\log u$ bits. This is possible because the maximum count of a given character in a block of size u, is u instances. Therefore, it can be stored in $\log u$ bits.

This expression can be simplified. First, the $|\Sigma|! \times |\Sigma|$ terms can be removed since they are smaller than $|\Sigma|^u$, and $u = \log |T|$. This leaves

$$= O(|\Sigma|^u \log |T| \log \log |T|)$$

Now, we can use another trick to improve our choice of u. Rather than just define $u = \log |T|$, we can be more specific and make it $u = \epsilon \log_{|\Sigma|} |T|$. This is still of logarithm order, so it does not break previous assumptions. However, it allows the first term $|\Sigma|^u$ to be simplified to $|T|^\epsilon$.

$$= O(|T|^\epsilon \log |T| \log \log |T|)$$

We can pick the ϵ to be whatever we want. If we make it $\epsilon = \frac{1}{2}$, then we now have a square root space complexity table.

From this, we have proven the table can be small if the block size is chosen wisely.